

15.3 External Searching and B-Trees

Consider the problem of maintaining a large collection of items that does not fit in main memory, such as a typical database. In this context, we refer to the secondary-memory blocks as *disk blocks*. Likewise, we refer to the transfer of a block between secondary memory and primary memory as a *disk transfer*. Recalling the great time difference that exists between main memory accesses and disk accesses, the main goal of maintaining such a collection in external memory is to minimize the number of disk transfers needed to perform a query or update. We refer to this count as the *I/O complexity* of the algorithm involved.

Some Inefficient External-Memory Representations

A typical operation we would like to support is the search for a key in a map. If we were to store n items unordered in a doubly linked list, searching for a particular key within the list requires n transfers in the worst case, since each link hop we perform on the linked list might access a different block of memory.

We can reduce the number of block transfers by storing the sequence in an array. A sequential search of an array can be performed using only $O(n/B)$ block transfers because of spatial locality of reference, where B denotes the number of elements that fit into a block. This is because the block transfer when accessing the first element of the array actually retrieves the first B elements, and so on with each successive block. It is worth noting that the bound of $O(n/B)$ transfers is only achieved when using an array of primitives in Java. For an array of objects, the array stores the sequence of references; the actual objects that are referenced are not necessarily stored near each other in memory, and so there may be n distinct block transfers in the worst case.

If a sequence is stored in *sorted* order within an array, a binary search performs $O(\log_2 n)$ transfers, which is a nice improvement. But we do not get significant benefit from block transfers because each query during a binary search is likely in a different block of the sequence. As usual, update operations are expensive for a sorted array.

Since these simple implementations are I/O inefficient, we should consider the logarithmic-time internal-memory strategies that use balanced binary trees (for example, AVL trees or red-black trees) or other search structures with logarithmic average-case query and update times (for example, skip lists or splay trees). Typically, each node accessed for a query or update in one of these structures will be in a different block. Thus, these methods all require $O(\log_2 n)$ transfers in the worst case to perform a query or update operation. But we can do better! We can perform map queries and updates using only $O(\log_B n) = O(\log n / \log B)$ transfers.

15.3.1 (a, b) Trees

To reduce the number of external-memory accesses when searching, we can represent our map using a multiway search tree (Section 11.5.1). This approach gives rise to a generalization of the $(2, 4)$ tree data structure known as the (a, b) tree.

An (a, b) tree is a multiway search tree such that each node has between a and b children and stores between $a - 1$ and $b - 1$ entries. The algorithms for searching, inserting, and removing entries in an (a, b) tree are straightforward generalizations of the corresponding ones for $(2, 4)$ trees. The advantage of generalizing $(2, 4)$ trees to (a, b) trees is that a parameterized class of trees provides a flexible search structure, where the size of the nodes and the running time of the various map operations depends on the parameters a and b . By setting the parameters a and b appropriately with respect to the size of disk blocks, we can derive a data structure that achieves good external-memory performance.

Definition of an (a, b) Tree

An (a, b) **tree**, where parameters a and b are integers such that $2 \leq a \leq (b + 1)/2$, is a multiway search tree T with the following additional restrictions:

Size Property: Each internal node has at least a children, unless it is the root, and has at most b children.

Depth Property: All the external nodes have the same depth.

Proposition 15.1: The height of an (a, b) tree storing n entries is $\Omega(\log n / \log b)$ and $O(\log n / \log a)$.

Justification: Let T be an (a, b) tree storing n entries, and let h be the height of T . We justify the proposition by establishing the following bounds on h :

$$\frac{1}{\log b} \log(n + 1) \leq h \leq \frac{1}{\log a} \log \frac{n + 1}{2} + 1.$$

By the size and depth properties, the number n'' of external nodes of T is at least $2a^{h-1}$ and at most b^h . By Proposition 11.6, $n'' = n + 1$. Thus,

$$2a^{h-1} \leq n + 1 \leq b^h.$$

Taking the logarithm in base 2 of each term, we get

$$(h - 1) \log a + 1 \leq \log(n + 1) \leq h \log b.$$

An algebraic manipulation of these inequalities completes the justification. ■

Search and Update Operations

We recall that in a multiway search tree T , each node w of T holds a secondary structure $M(w)$, which is itself a map (Section 11.5.1). If T is an (a, b) tree, then $M(w)$ stores at most b entries. Let $f(b)$ denote the time for performing a search in a map, $M(w)$. The search algorithm in an (a, b) tree is exactly like the one for multiway search trees given in Section 11.5.1. Hence, searching in an (a, b) tree T with n entries takes $O(\frac{f(b)}{\log a} \log n)$ time. Note that if b is considered a constant (and thus a is also), then the search time is $O(\log n)$.

The main application of (a, b) trees is for maps stored in external memory. Namely, to minimize disk accesses, we select the parameters a and b so that each tree node occupies a single disk block (so that $f(b) = 1$ if we wish to simply count block transfers). Providing the right a and b values in this context gives rise to a data structure known as the B-tree, which we will describe shortly. Before we describe this structure, however, let us discuss how insertions and removals are handled in (a, b) trees.

The insertion algorithm for an (a, b) tree is similar to that for a $(2, 4)$ tree. An overflow occurs when an entry is inserted into a b -node v , which becomes an illegal $(b + 1)$ -node. (Recall that a node in a multiway tree is a d -node if it has d children.) To remedy an overflow, we split node w by moving the median entry of w into the parent of w and replacing w with a $\lceil (b + 1)/2 \rceil$ -node w' and a $\lfloor (b + 1)/2 \rfloor$ -node w'' . We can now see the reason for requiring $a \leq (b + 1)/2$ in the definition of an (a, b) tree. Note that as a consequence of the split, we need to build the secondary structures $M(w')$ and $M(w'')$.

Removing an entry from an (a, b) tree is similar to what was done for $(2, 4)$ trees. An underflow occurs when a key is removed from an a -node w , distinct from the root, which causes w to become an illegal $(a - 1)$ -node. To remedy an underflow, we perform a transfer with a sibling of w that is not an a -node or we perform a fusion of w with a sibling that is an a -node. The new node w' resulting from the fusion is a $(2a - 1)$ -node, which is another reason for requiring $a \leq (b + 1)/2$.

Table 15.1 shows the performance of a map realized with an (a, b) tree.

Method	Running Time
get	$O\left(\frac{f(b)}{\log a} \log n\right)$
put	$O\left(\frac{g(b)}{\log a} \log n\right)$
remove	$O\left(\frac{g(b)}{\log a} \log n\right)$

Table 15.1: Time bounds for an n -entry map realized by an (a, b) tree T . We assume the secondary structure of the nodes of T support search in $f(b)$ time, and split and fusion operations in $g(b)$ time, for some functions $f(b)$ and $g(b)$, which can be made to be $O(1)$ when we are only counting disk transfers.

15.3.2 B-Trees

A version of the (a, b) tree data structure, which is the best-known method for maintaining a map in external memory, is called the “B-tree.” (See Figure 15.7.) A **B-tree of order d** is an (a, b) tree with $a = \lceil d/2 \rceil$ and $b = d$. Since we discussed the standard map query and update methods for (a, b) trees above, we restrict our discussion here to the I/O complexity of B-trees.

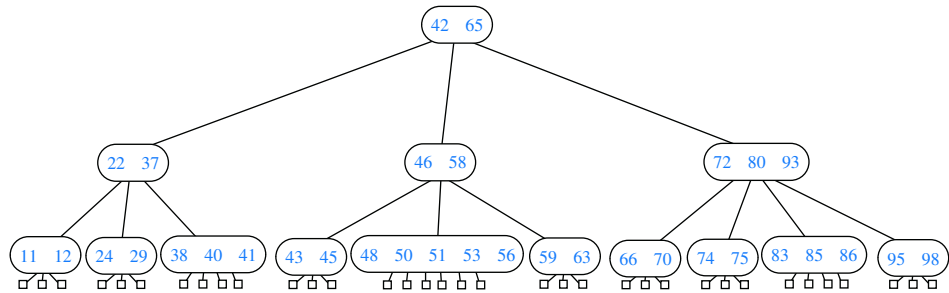


Figure 15.7: A B-tree of order 6.

An important property of B-trees is that we can choose d so that the d children references and the $d - 1$ keys stored at a node can fit compactly into a single disk block, implying that d is proportional to B . This choice allows us to assume that a and b are also proportional to B in the analysis of the search and update operations on (a, b) trees. Thus, $f(b)$ and $g(b)$ are both $O(1)$, for each time we access a node to perform a search or an update operation, we need only perform a single disk transfer.

As we have already observed above, each search or update requires that we examine at most $O(1)$ nodes for each level of the tree. Therefore, any map search or update operation on a B-tree requires only $O(\log_{\lceil d/2 \rceil} n)$, that is, $O(\log n / \log B)$, disk transfers. For example, an insert operation proceeds down the B-tree to locate the node in which to insert the new entry. If the node would **overflow** (to have $d + 1$ children) because of this addition, then this node is **split** into two nodes that have $\lfloor (d + 1)/2 \rfloor$ and $\lceil (d + 1)/2 \rceil$ children, respectively. This process is then repeated at the next level up, and will continue for at most $O(\log_B n)$ levels.

Likewise, if a remove operation results in a node **underflow** (to have $\lceil d/2 \rceil - 1$ children), then we move references from a sibling node with at least $\lceil d/2 \rceil + 1$ children or we perform a **fusion** operation of this node with its sibling (and repeat this computation at the parent). As with the insert operation, this will continue up the B-tree for at most $O(\log_B n)$ levels. The requirement that each internal node have at least $\lceil d/2 \rceil$ children implies that each disk block used to support a B-tree is at least half full. Thus, we have the following:

Proposition 15.2: A B-tree with n entries has I/O complexity $O(\log_B n)$ for search or update operation, and uses $O(n/B)$ blocks, where B is the size of a block.